

Core Java Interview Questions

Section 1: Java Basics & OOP

1. What is Java? Why is it called platform-independent?
 2. Explain JDK, JRE, and JVM differences.
 3. What are the main features of Java?
 4. What is the difference between JDK, JRE, and JVM memory management?.
 5. What is an object and a class? When do you use each?
 6. What are constructors? Can they be overloaded?
 7. What is the difference between default, parameterized, and private constructors?
 8. Explain static keyword. When should we use it?
 9. What are instance variables vs static variables?
 10. What is method overloading and overriding?
 11. What is abstraction? How is it achieved in Java?
 12. What is the difference between abstract class and interface?
 13. What is encapsulation? Why is it important?
 14. What is inheritance? Types of inheritance in Java?
 15. What is polymorphism? Explain compile-time vs runtime polymorphism.
 16. What is the difference between "==" and equals()?
 17. What is the difference between String, StringBuilder, and StringBuffer?
 18. What is a final keyword in Java? Where can it be applied?
 19. What is the difference between this and super keywords?
 20. What is the difference between early binding and late binding?
-

Section 2: Java Memory, Exceptions, and Flow Control (21–40)

- 21. What is the JVM memory structure? Explain Heap, Stack, and Method Area.**
 - 22. What is Garbage Collection? How does it work?**
 - 23. What is the difference between final, finally, and finalize()?**
 - 24. What are checked and unchecked exceptions? Examples?**
 - 25. What is the difference between throw and throws?**
 - 26. What is the difference between Error and Exception in Java?**
 - 27. What is a NullPointerException? How can it be avoided?**
 - 28. What is the difference between throw new Exception() and throw e?**
 - 29. What is a try-with-resources block? When to use it?**
 - 30. What is the difference between continue and break?**
 - 31. What is the difference between if-else and switch?**
 - 32. What is the difference between while, do-while, and for loops?**
 - 33. What are labeled loops? When do we use them?**
 - 34. What is the difference between shallow copy and deep copy?**
 - 35. What is type casting? Explain upcasting and downcasting.**
 - 36. What is the difference between primitive and wrapper classes?**
 - 37. What is autoboxing and unboxing?**
 - 38. What is the difference between volatile, transient, and synchronized keywords?**
 - 39. What is the difference between == and equals() for wrapper objects?**
 - 40. What is a Singleton class? How do you implement it?**
-

Section 3: Collections & Generics

- 41. What is the difference between ArrayList, LinkedList, Vector, and Stack?**
- 42. What is the difference between List, Set, and Map?**

- 43. What is the difference between HashMap, TreeMap, LinkedHashMap?
 - 44. What is the difference between HashSet, LinkedHashSet, TreeSet?
 - 45. What is fail-fast and fail-safe in Java Collections?
 - 46. What is the difference between Comparable and Comparator?
 - 47. What is the difference between Iterator and ListIterator?
 - 48. What is the difference between Enumeration and Iterator?
 - 49. What is the difference between synchronized collection and concurrent collection?
 - 50. What is the difference between Vector and ArrayList?
 - 51. What is the difference between Queue, PriorityQueue, and Deque?
 - 52. What is the difference between HashMap and Hashtable?
 - 53. What is the difference between identityHashMap and WeakHashMap?
 - 54. What is the difference between Map.keySet(), Map.values(), and Map.entrySet()?
 - 55. What is the difference between Array and ArrayList?
 - 56. What is the difference between Enumeration and Stream API in collections?
 - 57. What is the difference between generic and non-generic collections?
 - 58. What is the difference between bounded and unbounded wildcards in generics?
 - 59. What is the difference between List.of() and Arrays.asList()?
 - 60. What is the difference between Optional and null in Java?
-

Section 4: Multithreading & Concurrency

- 61. What is the difference between process and thread?
- 62. What is the difference between Runnable and Thread class?
- 63. What is thread lifecycle in Java?

- 64. What is the difference between `start()` and `run()` method in `Thread`?
 - 65. What is the difference between `sleep()` and `wait()`?
 - 66. What is the difference between `notify()` and `notifyAll()`?
 - 67. What is the difference between synchronized block and synchronized method?
 - 68. What is the difference between `ReentrantLock` and `synchronized`?
 - 69. What is deadlock? How to avoid it?
 - 70. What is livelock and starvation?
 - 71. What is the difference between `ExecutorService` and `ThreadPool`?
 - 72. What is the difference between `Callable` and `Runnable`?
 - 73. What is `Future` and `FutureTask`?
 - 74. What is `concurrentHashMap` and how it differs from `HashMap`?
 - 75. What is atomic variable? Explain `AtomicInteger`.
 - 76. What is the difference between `ThreadLocal` and normal variable?
 - 77. What is fork/join framework?
 - 78. What is the difference between synchronized collection and concurrent collection?
 - 79. What is `Phaser`, `CountDownLatch`, and `CyclicBarrier`?
 - 80. What is the difference between `scheduleAtFixedRate()` and `scheduleWithFixedDelay()`?
-

Section 5: Java 8+ Features

- 81. What is Lambda expression? When to use it?
- 82. What is the difference between Stream API and Collection API?
- 83. What is the difference between `map()` and `flatMap()` in streams?
- 84. What is the difference between `forEach()` and `map()`?
- 85. What is `Optional` in Java? Why is it used?

86.What is the difference between Predicate, Function, Supplier, and Consumer?

87.What is the difference between parallelStream() and stream()?

88.What is the difference between reduce() and collect() in streams?

89.What is the difference between method reference and lambda expression?

90.What is the difference between peek() and map() in streams?

Section 6: JVM, Classloading & Advanced Concepts

91.What is the difference between stack memory and heap memory?

92.What is the difference between strong, soft, weak, and phantom references?

93.What is the difference between Class.forName() and new operator?

94.What is the difference between compile-time and runtime polymorphism?

95.What is the difference between static binding and dynamic binding?

96.What is the difference between checked and unchecked exceptions?

97.What is the difference between Java memory model and heap memory?

98.What is the difference between Java 7 try-with-resources and Java 6 try-finally?

99.What is the difference between transient and volatile keywords?

100. What is the difference between final, immutable, and constant in Java?

Why these questions?

- These are core concepts every Java developer must know.
- Most interview filters fail candidates if they cannot answer OOP, collections, exceptions, multithreading, JVM concepts.
- Knowing these ensures a solid foundation and makes you confident in coding rounds.

Answers (THIS IS ONLY FOR IDEA)

This answer is provided only as a reference or idea. You can customize it further by referring to Niyaz Sir's notes, browsing reliable sources, or using AI tools. Please ensure you follow your own understanding and present it in your professional style, rather than memorizing or copying verbatim.

Section 1: Java Basics & OOP

Q1. What is Java? Why is it called platform-independent?

- **Answer:** Java is an object-oriented, high-level, general-purpose programming language.
- Platform-independent because Java code is compiled into **bytecode**, which runs on any system that has a **JVM**.
- **Example:**

```
System.out.println("Hello World");
```

The same .class file can run on Windows, Linux, Mac.

Q2. Explain JDK, JRE, and JVM differences.

- **JVM:** Executes bytecode, manages memory, platform-dependent.
- **JRE:** JVM + libraries, allows running Java programs.
- **JDK:** JRE + compiler + tools, used for development.
- **Note:** Forgetting JDK vs JRE causes compile/run confusion.

Q3. What are the main features of Java?

- Object-Oriented, Platform-Independent, Secure, Robust, Multithreading, High Performance.
- **Note:** People sometimes think Java is compiled to machine code; it's compiled to bytecode.

Q4. JDK, JRE, JVM memory differences

- JVM manages **stack, heap, method area**.
- JRE provides runtime libraries; JDK provides compiler.
- **Pitfall:** Assuming JRE alone can compile Java (it cannot).

Q5. Object vs Class

- Class = blueprint, Object = instance.
- **Example:**

```
class Car {}
```

```
Car c = new Car(); // c is an object
```

Q6. Constructors and overloading

- Special method to initialize objects. Overloading allowed.

```
class Car {
```

```
    Car() {} // default
```

```
    Car(String name) {} // parameterized
```

```
}
```

Q7. Default, parameterized, private constructors

- Default: no args, auto-provided if no constructor.
- Parameterized: custom initialization.
- Private: restrict object creation (Singleton pattern).

Q8. Static keyword

- Belongs to class, not object.
- **Example:**

```
class Demo {
```

```
static int count;  
}
```

- Use for constants, utility methods.
- **Pitfall:** Static cannot access instance variables directly.

Q9. Instance vs Static Variables

- Instance: per object
- Static: shared among all objects

```
class Demo {  
    int a; // instance  
    static int b; // static  
}
```

Q10. Method overloading vs overriding

- Overloading: compile-time, same name, different params
- Overriding: runtime, subclass provides specific behavior
- **Note:** private methods cannot be overridden.

Q11. Abstraction in Java

- Hiding implementation
- Achieved via **abstract class** or **interface**
- **Example:**

```
abstract class Shape {  
    abstract void draw();  
}
```

Q12. Abstract class vs Interface

- Abstract: can have constructors, fields
- Interface: multiple inheritance allowed, methods abstract by default (pre-Java 8)
- **Note:** Multiple inheritance via classes not allowed, only via interfaces.

Q13. Encapsulation

- Wrap data + methods; control access via **private + getters/setters**
- **Example:**

```
class Person {  
    private String name;  
    public String getName() { return name; }  
}
```

Q14. Inheritance & types

- Reuse code via extends
- Types: Single, Multilevel, Hierarchical
- **Note:** Multiple inheritance not allowed for classes (diamond problem).

Q15. Polymorphism

- Compile-time: overloading
- Runtime: overriding
- Same interface, multiple forms.

Q16. == vs equals()

- == compares references, equals() compares content
- **Note:** String s1="abc"; s2=new String("abc"); s1==s2 → false, s1.equals(s2) → true

Q17. String vs StringBuilder vs StringBuffer

- String: immutable
- StringBuilder: mutable, not synchronized
- StringBuffer: mutable, synchronized (thread-safe)

Q18. final keyword

- Variable: cannot reassign
- Method: cannot override
- Class: cannot extend

Q19. this vs super

- this: current object
- super: parent class object or method

Q20. Early vs late binding

- Early: compile-time (overloading, static)
 - Late: runtime (overriding, dynamic)
 - **Note:** Static methods are always early bound.
-

Section 2: Memory, Exceptions & Flow Control

Q21. What is JVM memory structure? Explain Heap, Stack, and Method Area.

- **Heap:** Stores objects at runtime. Garbage-collected.
- **Stack:** Stores method calls, local variables, and references. LIFO structure.
- **Method Area (PermGen/Metaspace):** Stores class metadata, constants, and static variables.
- **Note:** Local variables are in stack, but object data is in heap. Modifying object in one method affects references in another method.

Q22. What is Garbage Collection? How does it work?

- **Automatic memory management** in Java. JVM identifies objects with no references and removes them.
- **Note:** finalize() method exists but is **deprecated** in modern Java.

System.gc(); // Suggests GC, not guaranteed

Q23. Difference between final, finally, and finalize()

Keyword Purpose

final Prevent changes (variable, method, class)

finally Block executed after try-catch (always executes)

Keyword Purpose

`finalize()` Method called before GC to clean resources (deprecated in Java 9+)

Q24. Checked vs Unchecked exceptions

- **Checked:** Must be handled (`IOException`, `SQLException`)
- **Unchecked:** Runtime exceptions (`NullPointerException`, `ArithmeticException`)

Q25. throw vs throws

- **throw:** Used to explicitly throw an exception
- **throws:** Declares exception in method signature

```
void method() throws IOException { ... }
```

```
throw new IOException("Error");
```

Q26. Difference between Error and Exception

- **Error:** JVM-level problems, cannot recover (`OutOfMemoryError`)
- **Exception:** Application-level, can be handled

Q27. NullPointerException. How to avoid it?

- Occurs when calling a method/field on null object
- **Avoid:** null checks, `Optional` class

```
String s = null;
```

```
if(s != null) System.out.println(s.length());
```

Q28. Difference between throw new Exception() and throw e

- `throw new Exception():` Creates new exception object
- `throw e:` Re-throws existing exception (useful in catch)

Q29. try-with-resources block

- Auto-closes resources implementing **AutoCloseable**
- **Example:**

```
try(FileReader fr = new FileReader("file.txt")) {
```

```
// read file  
} // Auto-closed
```

Q30. Difference between continue and break

- **break:** exit loop entirely
- **continue:** skip current iteration and continue next

Q31. Difference between if-else and switch

- **if-else:** evaluates boolean expressions, works for ranges
- **switch:** evaluates discrete values, faster in some cases

Q32. while, do-while, for loops

- **while:** condition first
- **do-while:** executes at least once
- **for:** initialization, condition, increment

Q33. Labeled loops

- Allows breaking/continuing outer loops

```
outer: for(int i=0;i<5;i++){  
    for(int j=0;j<5;j++){  
        if(j==2) break outer;  
    }  
}
```

Q34. Shallow copy vs Deep copy

- **Shallow copy:** copies object reference (changes reflect in original)
- **Deep copy:** copies object data (original unaffected)

Q35. Type casting – upcasting vs downcasting

- **Upcasting:** subclass → superclass (safe, implicit)
- **Downcasting:** superclass → subclass (explicit, may throw `ClassCastException`)

```
Animal a = new Dog(); // upcasting
```

Dog d = (Dog) a; // downcasting

Q36. Primitive vs Wrapper classes

- Primitive: int, char, double (faster, memory-efficient)
- Wrapper: Integer, Character, Double (used in Collections)

Q37. Autoboxing and unboxing

- Autoboxing: primitive → wrapper
- Unboxing: wrapper → primitive

Integer i = 10; // autoboxing

int j = i; // unboxing

Q38. Difference between volatile, transient, synchronized

- **volatile:** ensures visibility across threads
- **transient:** excludes field from serialization
- **synchronized:** thread-safety for blocks/methods

Q39. Difference between == and equals() for wrapper objects

- == checks reference
- equals() checks value
- **Note:** Integer i1=128, i2=128; i1==i2 → false (outside cache -128 to 127)

Q40. Singleton class – How to implement

- Restrict object creation to one instance
- **Example:**

```
class Singleton {  
    private static Singleton instance;  
    private Singleton(){}  
    public static Singleton getInstance(){  
        if(instance==null) instance=new Singleton();  
        return instance;  
    }  
}
```

```
}  
}
```

- **Note:** Make it thread-safe using synchronized or Bill Pugh Singleton.
-

Section 3: Collections & Generics

Q41. Difference between ArrayList, LinkedList, Vector, and Stack

Collection	Underlying Structure	Synchronization	Performance	Notes
ArrayList	Dynamic array	No	Fast random access	Best for frequent get() operations
LinkedList	Doubly linked list	No	Slow random access, fast insert/delete	Can act as Queue/Deque
Vector	Dynamic array	Yes	Slower due to synchronization	Legacy class
Stack	Vector	Yes	LIFO	Use Deque instead in modern Java

Note: Vector is thread-safe, but still considered legacy. Modern Java prefers **ArrayList + Collections.synchronizedList()** or **CopyOnWriteArrayList**.

Q42. Difference between List, Set, and Map

- **List:** Ordered, allows duplicates (ArrayList, LinkedList)
- **Set:** Unordered, no duplicates (HashSet, TreeSet)
- **Map:** Key-value pairs, no duplicate keys (HashMap, TreeMap)

```
List<String> list = new ArrayList<>();
```

```
Set<String> set = new HashSet<>();
```

```
Map<String, Integer> map = new HashMap<>();
```

Note: HashSet internally uses HashMap.

Q43. Difference between HashMap, TreeMap, LinkedHashMap

- **HashMap:** Unordered, allows null key, O(1) get/put
 - **TreeMap:** Sorted order, no null key, O(log n) operations
 - **LinkedHashMap:** Maintains insertion order
-

Q44. Difference between HashSet, LinkedHashSet, TreeSet

- **HashSet:** Unordered, fastest
 - **LinkedHashSet:** Insertion order
 - **TreeSet:** Sorted order, implements NavigableSet
-

Q45. Fail-fast vs Fail-safe

- **Fail-fast:** Throws ConcurrentModificationException if collection is modified during iteration (ArrayList, HashMap)
- **Fail-safe:** Doesn't throw, iterates over snapshot (ConcurrentHashMap, CopyOnWriteArrayList)

```
List<Integer> list = new ArrayList<>(List.of(1,2,3));
```

```
for(Integer i : list) list.add(4); // ConcurrentModificationException
```

Q46. Comparable vs Comparator

- **Comparable:** Defines natural order inside class (compareTo)
- **Comparator:** External class to define custom order (compare method)

```
class Person implements Comparable<Person> {
```

```
    int age;
```

```
public int compareTo(Person p){ return this.age - p.age; }  
}
```

```
Comparator<Person> nameComparator = (p1,p2)-  
>p1.name.compareTo(p2.name);
```

Q47. Iterator vs ListIterator

Feature	Iterator	ListIterator
Direction	Forward	Forward & Backward
Methods	hasNext(), next(), remove()	hasPrevious(), previous(), add(), set()
Collection	All collections	Only List

Q48. Enumeration vs Iterator

- Enumeration: Legacy, only read, methods hasMoreElements(), nextElement()
 - Iterator: Modern, can remove, fail-fast
-

Q49. Synchronized collection vs Concurrent collection

- Synchronized (Collections.synchronizedList()): Thread-safe, locks entire collection
 - Concurrent (CopyOnWriteArrayList, ConcurrentHashMap): Efficient for reads, better concurrency
-

Q50. Vector vs ArrayList

- Vector: Synchronized, slower
 - ArrayList: Unsynchronized, faster
-

Q51. Queue vs PriorityQueue vs Deque

- Queue: FIFO
 - PriorityQueue: Elements ordered by priority (Comparator)
 - Deque: Double-ended queue (add/remove from both ends)
-

Q52. HashMap vs Hashtable

- HashMap: Non-synchronized, allows null key/value
 - Hashtable: Synchronized, no null key/value
-

Q53. IdentityHashMap vs WeakHashMap

- IdentityHashMap: Compares keys by reference (==)
 - WeakHashMap: Keys are weakly referenced, GC removes entries
-

Q54. Map.keySet(), Map.values(), Map.entrySet()

- keySet(): Set of keys
 - values(): Collection of values
 - entrySet(): Set of Map.Entry<K,V>, allows iteration over key-value pairs efficiently
-

Q55. Array vs ArrayList

Feature	Array	ArrayList
Size	Fixed	Dynamic
Type	Primitive & Objects	Objects only
Performance	Faster	Slight overhead

Q56. Enumeration vs Stream API in collections

- Enumeration: Legacy, only read

- Stream API: Functional programming, allows **map, filter, reduce, parallel streams**
-

Q57. Generic vs Non-generic collections

- Generic: Type-safe, avoids ClassCastException
- Non-generic: Requires casting, runtime risk

```
List<String> list = new ArrayList<>(); // generic
```

Q58. Bounded vs Unbounded wildcards

- ? extends T → Upper-bounded (read-only)
- ? super T → Lower-bounded (write allowed)
- ? → Unbounded

```
List<? extends Number> nums; // can read Number
```

```
List<? super Integer> ints; // can add Integer
```

Q59. List.of() vs Arrays.asList()

- List.of() → immutable, cannot add/remove
 - Arrays.asList() → fixed-size, allows set()
-

Q60. Optional vs null

- Optional: Avoid NullPointerException, functional-style handling

```
Optional<String> opt = Optional.ofNullable(null);
```

```
opt.ifPresent(System.out::println);
```

- **Note:** Optional should not be used for fields, only method returns.
-

Section 4: Multithreading & Concurrency

Q61. Difference between process and thread

- **Process:** Independent program with its own memory space
 - **Thread:** Lightweight, shares memory of process
 - **Note:** Multiple threads in one process share heap, but have separate stack.
-

Q62. Runnable vs Thread class

- **Runnable:** Functional interface, pass to Thread, better for multiple inheritance
- **Thread:** Can be extended, contains run() method

```
class MyTask implements Runnable {  
    public void run(){ System.out.println("Running"); }  
}  
  
new Thread(new MyTask()).start();
```

Q63. Thread lifecycle in Java

- **New → Runnable → Running → Waiting/Blocked → Terminated**
 - **Note:** Thread.sleep() moves thread to **Timed Waiting**, not Runnable.
-

Q64. start() vs run() method in Thread

- **start():** Creates a new thread and calls run()
- **run():** Executes in current thread, no new thread created

```
Thread t = new Thread(() -> System.out.println("Thread"));  
t.start(); // new thread  
t.run(); // main thread
```

Q65. sleep() vs wait()

Feature	<code>sleep()</code>	<code>wait()</code>
Belongs to	Thread	Object
Releases lock	No	Yes
Use	Pause thread	Inter-thread communication

Q66. `notify()` vs `notifyAll()`

- **`notify()`**: Wakes one waiting thread
 - **`notifyAll()`**: Wakes all waiting threads
 - Must be called from synchronized block.
-

Q67. Synchronized block vs synchronized method

- **Method**: Locks entire object or class (for static)
 - **Block**: Locks only critical section
 - **Note**: Synchronized on different objects → no lock interference
-

Q68. `ReentrantLock` vs `synchronized`

- **`ReentrantLock`**: More flexible (`lockInterruptibly`, `tryLock`, fairness)
- **`synchronized`**: Simpler, less code
- Example:

```
ReentrantLock lock = new ReentrantLock();
```

```
lock.lock();
```

```
try { /* critical section */ }
```

```
finally { lock.unlock(); }
```

Q69. Deadlock & how to avoid

- Occurs when two threads wait on each other's resources

- Avoid: Lock ordering, tryLock(), reduce nested locks
-

Q70. Livelock & starvation

- **Livelock:** Threads active but cannot progress (polite threads yield repeatedly)
 - **Starvation:** Low-priority threads never get CPU
-

Q71. ExecutorService vs ThreadPool

- **ThreadPool:** Low-level thread management
- **ExecutorService:** High-level API to manage thread pools

```
ExecutorService service = Executors.newFixedThreadPool(5);  
service.submit(() -> System.out.println("Task"));  
service.shutdown();
```

Q72. Callable vs Runnable

- **Runnable:** no return, cannot throw checked exception
- **Callable:** returns value, can throw exception

```
Callable<Integer> task = () -> 123;  
Future<Integer> result = executor.submit(task);  
System.out.println(result.get());
```

Q73. Future & FutureTask

- **Future:** Represents result of async computation
 - **FutureTask:** Implements Runnable + Future
-

Q74. ConcurrentHashMap vs HashMap

- **ConcurrentHashMap:** thread-safe, segmented locking

- HashMap: not thread-safe, ConcurrentModificationException if modified during iteration
-

Q75. Atomic variable – example AtomicInteger

- Provides lock-free thread-safe operations

```
AtomicInteger count = new AtomicInteger(0);  
count.incrementAndGet();
```

Q76. ThreadLocal vs normal variable

- ThreadLocal: Each thread has its own copy
- Normal variable: Shared between threads (prone to race conditions)

```
ThreadLocal<Integer> local = ThreadLocal.withInitial(() -> 0);
```

Q77. Fork/Join framework

- Divide-and-conquer parallelism
 - RecursiveTask or RecursiveAction used for splitting tasks
-

Q78. Synchronized collection vs concurrent collection

- Synchronized: Locks entire collection
 - Concurrent: Fine-grained locking, better concurrency (CopyOnWriteArrayList, ConcurrentHashMap)
-

Q79. Phaser, CountdownLatch, CyclicBarrier

Class	Purpose
CountDownLatch	Wait until count reaches zero, one-time
CyclicBarrier	Wait for all threads, reusable

Class	Purpose
Phaser	More flexible barrier with phases

Q80. `scheduleAtFixedRate()` vs `scheduleWithFixedDelay()`

- **`scheduleAtFixedRate`:** Executes at fixed intervals regardless of task duration
 - **`scheduleWithFixedDelay`:** Executes after fixed delay after previous task completes
-

Section 5: Java 8+ Features

Q81. What is a Lambda expression? When to use it?

- **Lambda:** Anonymous function, used to implement functional interfaces.
- **Use:** Simplifies code, replaces anonymous classes.

```
Runnable r = () -> System.out.println("Lambda thread");
new Thread(r).start();
```

- **Note:** Lambdas cannot access non-final local variables from enclosing scope.
-

Q82. Stream API vs Collection API

- **Collection API:** Iterative, imperative style
- **Stream API:** Functional, supports map, filter, reduce, parallel operations

```
List<Integer> nums = List.of(1,2,3,4);
nums.stream().filter(n->n%2==0).forEach(System.out::println);
```

Q83. `map()` vs `flatMap()` in streams

- **`map()`:** One-to-one mapping
- **`flatMap()`:** One-to-many mapping, flattens nested structures

```
List<List<Integer>> nested = List.of(List.of(1,2), List.of(3,4));  
nested.stream().flatMap(List::stream).forEach(System.out::println);
```

Q84. `forEach()` vs `map()`

- **`forEach()`**: Terminal operation, returns void
 - **`map()`**: Intermediate, returns transformed stream
-

Q85. Optional in Java & why used

- Represents a container that may/may not contain a value
- Avoids **`NullPointerException`**

```
Optional<String> name = Optional.ofNullable(null);  
name.ifPresent(System.out::println);
```

- **Note**: Optional should be used for return types, not fields.
-

Q86. Predicate, Function, Supplier, Consumer

Interface	Purpose	Example
<code>Predicate<T></code>	Tests condition, returns boolean	<code>Predicate<Integer> p = n->n>10;</code>
<code>Function<T,R></code>	Transforms input	<code>Function<String,Integer> f = s->s.length();</code>
<code>Supplier<T></code>	Supplies value	<code>Supplier<Date> s = ()->new Date();</code>
<code>Consumer<T></code>	Consumes input, no return	<code>Consumer<String> c = System.out::println;</code>

Q87. `parallelStream()` vs `stream()`

- **`stream()`**: Sequential processing
- **`parallelStream()`**: Uses multiple threads for parallel processing

- **Note:** Thread-safety issues with shared mutable data.
-

Q88. `reduce()` vs `collect()` in streams

- **`reduce()`:** Combines elements into a single value (immutable)

```
int sum = List.of(1,2,3).stream().reduce(0, Integer::sum);
```

- **`collect()`:** Collects elements into a container (List, Set, Map)

```
List<String> result = List.of("a","b").stream().collect(Collectors.toList());
```

Q89. Method reference vs Lambda expression

- **Lambda:** `(x) -> System.out.println(x)`
 - **Method reference:** `System.out::println`
 - **Note:** Can only use method reference if signature matches functional interface.
-

Q90. `peek()` vs `map()` in streams

- **`map()`:** Transforms elements, intermediate
- **`peek()`:** Mainly for debugging, does not transform elements

```
List<Integer> nums = List.of(1,2,3);
```

```
nums.stream().peek(System.out::println).map(n->n*2).forEach(System.out::println);
```

Section 6: JVM, Classloading & Advanced Concepts

Q91. Difference between stack memory and heap memory

- **Stack:** Stores method calls, local variables; LIFO; automatically cleared
- **Heap:** Stores objects; garbage collected; shared among threads

- **Note:** Modifying an object in heap affects all references; primitives in stack are independent.
-

Q92. Strong, Soft, Weak, and Phantom references

Reference Type	GC Behavior	Use Case
Strong	Never collected until no reference	Normal references
Soft	Collected only if memory low	Caching
Weak	Collected anytime	WeakHashMap
Phantom	After finalize, before GC	Cleanup actions

Q93. Class.forName() vs new operator

- **new:** Compile-time class loading
- **Class.forName():** Runtime loading, executes static blocks

`Class.forName("MyClass");` // loads class dynamically

Q94. Compile-time vs runtime polymorphism

- **Compile-time:** Method overloading, operator overloading (resolved at compile time)
 - **Runtime:** Method overriding, dynamic dispatch (resolved at runtime)
-

Q95. Static binding vs dynamic binding

- **Static binding:** Compiler resolves (private, static, final methods)
 - **Dynamic binding:** JVM resolves (overridden methods)
-

Q96. Checked vs unchecked exceptions

- **Checked:** Must handle (IOException, SQLException)

- **Unchecked:** Runtime (NullPointerException, ArithmeticException)
-

Q97. Java Memory Model vs heap memory

- **Heap:** Physical memory storage for objects
 - **Java Memory Model (JMM):** Defines how threads interact with memory, visibility, and atomicity
 - **Note:** Without volatile or synchronized, threads may see stale data.
-

Q98. Java 7 try-with-resources vs Java 6 try-finally

- **Java 6:** Manual resource close in finally
- **Java 7:** Auto-close using AutoCloseable, less error-prone

```
try(FileReader fr = new FileReader("file.txt")) { }
```

Q99. transient vs volatile keywords

- **transient:** Field not serialized
 - **volatile:** Field visibility across threads, prevents caching in registers
 - **Note:** volatile does **not** make compound operations atomic.
-

Q100. final, immutable, constant in Java

- **final variable:** Cannot be reassigned
- **Immutable object:** State cannot change after creation (String, Integer)
- **Constant:** Typically public static final field

```
final int x = 10;
```

```
public static final double PI = 3.14;
```

```
String s = "immutable";
```